BIG DATA MINING AND ANALYTICS
ISSN 2096-0654 01/18 pp341–359
DOI: 10.26599/BDMA.2025.9020047
Volume 9, Number 2, April 2026

Numerical Constraint-Aware Dense Retrieval with Two-Phase Contrastive Learning

Meng Wang*, Yisong Wang, and Feifan Wu

Abstract: Retrieval-Augmented Generation (RAG) enhances Large Language Models (LLMs) by integrating external knowledge, leading to significant improvements in both factual accuracy and task performance. However, existing dense retrievers face considerable challenges when handling numerical constraints, particularly in queries requiring precise filtering conditions. To systematically explore these issues, we introduce Numerical Constraint Question (NumConQ), a comprehensive multi-domain benchmark dataset that contains more than 6500 queries covering healthcare, finance, education, sports, and movies. Empirical analysis reveals that state-of-the-art dense retrievers achieve only 16.3% accuracy in numerical constraint satisfaction, significantly underperforming relative to their semantic matching capabilities. To address these limitations, we propose Numerical Constraint-aware Retriever (NC-Retriever), which features: (1) a two-phase contrastive learning framework that combines in-batch negative samplings with progressively introduced hard negatives, and (2) a hybrid numerical representation scheme for consistent tokenization. Extensive experiments show that NC-Retriever achieves a relative improvement of 65.84% in recall@10 and a 78.28% increase in precision@10 compared to current state-of-the-art methods. The code and benchmark dataset are available at https://github.com/Tongji-KGLLM/NumConQ.

Key words: dense retriever; benchmark dataset; contrastive learning; numerical constraint query; Retrieval-augmented Generation (RAG)

1 Introduction

Retrieval-Augmented Generation (RAG)[1] has emerged as a powerful paradigm for knowledge-intensive tasks, integrating Large Language Models (LLMs) with external data repositories to enhance both factual accuracy and contextual depth. A key step in the RAG process is the retrieval, which identifies related documents from a vast corpus; the generator then leverages these retrieved documents to produce the final outputs[2]. Recent advances in dense retrieval, which encodes both queries and documents into high-dimensional embedding vectors[3], have substantially improved performance in a variety of open-domain tasks. However, existing dense retrievers predominantly focus on semantic matching and frequently struggle to account for numerical constraints stated in user queries[4]. Numerical constraint queries pose unique challenges that differ from purely semantic or topical queries. In many real-world scenarios—ranging from healthcare records analysis[5, 6] to financial audits—queries often contain strict numerical conditions, such as “over 65 years old”, “revenue less than 10 million”, or “release in 2009”. Meeting these

• Meng Wang and Yisong Wang are with the College of Design and Innovation, Tongji University, Shanghai 200092, China. E-mail: mengwangtj@tongji.edu.cn; 2433521@tongji.edu.cn.
• Feifan Wu is with School of Computer Science and Engineering, Southeast University, Nanjing 211189, China. E-mail: 220224380@seu.edu.cn.
* To whom correspondence should be addressed.
Manuscript received: 2025-02-15; revised: 2025-04-23; accepted: 2025-04-28

© The author(s) 2026. The articles published in this open access journal are distributed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/).

constraints requires understanding the semantic context and enforcing precise filtering based on numerical values. Standard dense retrievers, trained primarily on vast unstructured text corpora, tend to emphasize conceptual similarities (e.g., "film genre" or "country") while overlooking whether a retrieved document meets the exact numerical requirement specified in the query[7]. As illustrated in Fig. 1, when the query specifies that the "average salary is lower than 6 183 974 USD", the high performing Beijing Academy of Artificial Intelligence (BAAI) General Embedding M3 (BGE-M3) retriever[8] retrieves documents that include leagues with salaries of 6 650 000 USD (exceeding the threshold by 7.54%) and 10 710 000 USD (exceeding the threshold by 73.19%). This highlights the insufficient sensitivity of existing methods to numerical boundaries.

Current approaches exhibit three key limitations: (1) inconsistent tokenization for multi-digit numbers, (2) attention bias toward textual features, and (3) lack of numerical constraint-aware training objectives.

Against this backdrop, we construct Numerical Constraint Question (NumConQ), a multi-domain benchmark dataset, comprising 6577 numerical constraint queries across healthcare, finance, education, sports, and movies, paired with 5362 natural language documents. In each domain, we transform structured entries into textual descriptions, ensuring that every query includes at least one numerical constraint. Based on this dataset, we systematically evaluate five widely adopted dense retrievers: Contriever[9], TAS-B[10], DPR[11], BGE-M3[8], and jina-embeddings-v2-base-en of JinaAI[12]. Our experimental results reveal that even the best performing retriever, BGE-M3, achieves Recall@10 and Precision@10 scores of merely 40.37% and 10.45%, respectively, highlighting the ongoing challenges of accurately handling numerical constraints within dense retrievers.

To capture numerical constraints more effectively, we construct hard negative examples by subtly modifying the numerical value in queries. During training, we apply both in-batch negative samplings[13] and these hard negatives, but defer to introducing the hard negatives until the mid-to-late stages. This phased approach helps the retriever first grasp general semantic cues, and then tackle the more demanding numerical-based distinctions, leading to a deeper understanding of numerical information within the retriever. Furthermore, we observe that higher-digit numbers are frequently segmented into multiple subword tokens in inconsistent patterns during tokenization, hindering the model's comprehension of the magnitude and positional context. To address this, we implement a hybrid numerical representation that retains small numbers in their original digital forms while converting larger numbers into word-based expressions. This approach mitigates inconsistent tokenization and provides the model with a more granular grasp of numerical values, ultimately improving the retrieval performance on numerical constraint queries.

The key contributions of this paper are as follows:

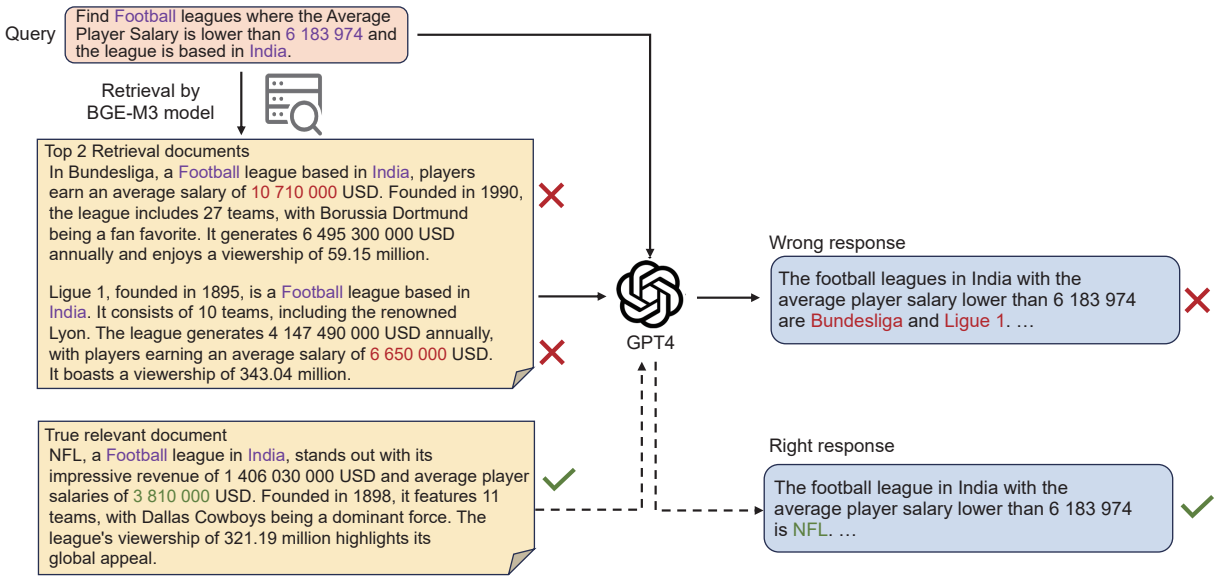


Fig. 1 Example of dense retriever failure when handling a numerical constraint query.

Multi-domain numerical benchmark: We introduce NumConQ, a multi-domain retrieval dataset consisting of 6577 text-based queries with numerical constraints and 5362 natural language documents, spanning healthcare, finance, education, sports, and movies. This benchmark systematically evaluates how well retrievers handle numerical conditions.

Deep analysis of retrievers' capabilities in handling numerical constraint: We investigate the shortcomings of current dense retrievers when dealing with numerical constraint, pinpointing factors such as tokenization inconsistencies and a lack of targeted training data that hinder accurate numerical filtering.

Phased learning and hybrid representation for numeric-constrained retrieval: We propose a two-pronged approach to improve retrieval performance: a phased contrastive learning framework incorporating hard negatives for numerical constraints and a hybrid numerical representation enhancing tokenization consistency and numerical understanding.

2 Related Work

This section briefly reviews related work in numerical constraint queries and dense retrievers.

2.1 Numerical constraint query

Semantic search has made significant progress due to advances in neural network architectures. However, handling numerical data remains a challenge for proposed methods.

Qsearch[14] pioneers the field as an end-to-end system for answering numerical constraint queries by leveraging deep neural networks to extract numerical values from text and transform them into semantic structures. The system introduces the use of Semantic Role Labeling (SRL) to classify words into entities, quantities, or context, enabling the extraction of numerical fact triples. Additionally, a novel statistical matching method is employed to retrieve and rank relevant answers efficiently.

Building on this foundation, Faith[15] extends the capabilities of numerical constraint query systems to temporal question answering across heterogeneous data sources. Faith addresses the challenge of implicit temporal constraints by transforming them into explicit values through intermediate questions and recursive system calls. This approach facilitates accurate and context-aware handling of temporal queries, showcasing the potential for adapting numerical constraint query methodologies to more complex scenarios.

To enhance the ability of LLMs to address numerical queries, Han et al.[16] tackled the limitations of LLMs in understanding numerical constraint queries within financial reports. The XBRL agent framework is introduced to enhance LLM performance by integrating external tools, effectively addressing the nuances of numerical reasoning in financial contexts. Furthermore, a dedicated dataset, the numeric type query task for XBRL reports, is proposed to benchmark and drive progress in this domain. Biswal et al.[17] introduced the Table-Augmented Generation (TAG) method, which combines LLMs and database systems to process complex natural language queries, including those with numerical constraints.

Together, these systems demonstrate a progressive refinement and expansion of numerical query capabilities on structured knowledge for the Question-Answer (QA) task of the knowledge base, where querying numerical knowledge is easier compared to unstructured documents. However, structured knowledge bases are much more difficult to obtain and maintain compared to unstructured documents, which allows our methods to leverage a broader range of numerical knowledge.

2.2 Dense retriever

Dense retrievers have gained significant attention in recent years due to their superior performance over traditional sparse retrieval methods, especially in scenarios requiring semantic understanding. The retrieval of relevant information from large corpora is a foundational task in Information Retrieval (IR). Traditional sparse retrieval methods, such as Term Frequency-Inverse Document Frequency (TF-IDF) and Best Matching 25 (BM25), rely on exact term matching, which often struggles with semantic similarity. Dense retrievers address this limitation by encoding queries and documents into dense vector representations, enabling similarity computation in a shared embedding space.

Dense encoders typically leverage pre-trained models to obtain rich contextual vector representations. There are two prevalent architectural paradigms for dense encoders: dual-encoder and cross-encoder frameworks. The dual-encoder architecture employs two distinct encoders to separately encode queries and documents,

subsequently computing semantic similarity between the query and document vectors. The cross-encoder architecture, conversely, concatenates the query and document as input, utilizing a single encoder to model query-document interactions and capture fine-grained semantic matching features directly. While cross-encoder architectures exhibit superior capability in capturing nuanced semantic features compared to dual-encoders, the latter allows for document pre-encoding and indexing, resulting in greater computational efficiency. Given the substantial volume of documents typically presented in naive RAG systems, dual-encoder architectures are predominantly employed for large-scale retrieval, while cross-encoders are commonly utilized as re-ranking mechanisms.

DPR[11] is a straightforward bi-encoder model that can learn embeddings from just a small dataset of questions and passages, enabling retrieval using solely dense representations. It employs positive examples from the QA dataset and challenging negative examples from BM25 for discriminative training. Xiong et al.[18] analyzed previous dense retrievers and identified that local negative sampling is a bottleneck in the performance of dense retrieval. Based on this, Approximate Negative Contrastive Estimation (ANCE) is proposed. Unlike random or in-batch local negatives, ANCE generates global negatives by leveraging the optimized dense retrieval model to retrieve from the whole corpus, therefore ensures that the distribution of negative samples during training aligns with the distribution of irrelevant documents during testing. Contriever[9] addresses the challenge of improving dense retrieval models in low-resource settings by proposing the use of contrastive learning as a method for training unsupervised dense retrievers, demonstrating that contrastive learning enhances the performance of these models in various retrieval settings. Recently, LARMOR[19] is an unsupervised method that leverages large language models to generate pseudo-relevant signals for selecting and ranking dense retrievers. KaLM-Embedding[20] integrates the multilingual knowledge of LLMs with dynamic training strategies, providing a robust semantic understanding foundation for RAG systems.

Despite the superior performance of dense retrievers in semantic understanding, they face challenges when handling numerically sensitive queries. Numerical-sensitive queries often require precise matching and accurate retrieval of detailed numerical information, whereas dense retrievers primarily rely on latent space semantic similarity, which may not capture fine-grained numerical differences. This is because dense models focus more on complex contextual and semantic relationships, rather than on exact numerical precision.

3 NumConQ Benchmark

In this section, we provide a detailed description of NumConQ and its construction to evaluate the capacity of dense retrievers to handle numerical constraint queries.

3.1 Problem definition

The task of numerical constraint query retrieval involves processing queries with numerical constraints and retrieving documents that satisfy both these constraints and any specified non-numerical ones. Queries and documents are typically expressed in natural language to reflect the real-world usage of dense retrievers.

Let a text-based query Q define constraint set C, decomposed as

$$C = C_{\text{num}} \bigcup C_{\text{other}}$$

where

- $C_{\text{num}} = \{(A_1, \text{op}_1, v_1), (A_2, \text{op}_2, v_2), \ldots, (A_n, \text{op}_n, v_n)\}$ denotes numerical constraints with:
  - A_i: Numerical attribute within the documents,
  - $\text{op}_i \in \{=, <, >, \geqslant, \geqslant\}$: Comparison operator,
  - $v_i \in \mathbf{R}$: Reference value.
- $C_{\text{other}} = \{c'_1, c'_2, \ldots, c'_m\}$ denotes non-numerical constraints (textual, categorical, etc.).

Given document collection D, retrieve all $d \in D$ satisfying

$$\forall (A_i, \text{op}_i, v_i) \in C_{\text{num}}: \ d[A_i] \times \text{op}_i \times v_i \Rightarrow \text{True},$$
$$\forall c'_j \in C_{\text{other}}: \ d \models c'_j,$$

where

- $d[A_i]$: Value of attribute A_i in document d,
- $\models$: Constraint satisfaction relation.

Numerical constraint queries can be categorized based on the nature of their constraints. One type is the exact match query, where a numerical value must precisely match a specified number, such as "Which athlete is 185cm tall?". Another is the range match query, where the value falls within a specified range, such as inclusive ranges like "Find athletes aged 25 to

30" or open-ended constraints like "age over 28" or "height less than 180 cm".

The query may involve a single numerical constraint or multiple constraints that must be satisfied simultaneously, such as "age less than 30 and height higher than 175 cm". The complexity increases with multiple constraints, influencing retrieval performance and accuracy.

3.2 Dataset construction

Existing retrieval datasets, such as ReQA[21], are primarily designed to focus on semantic information rather than on numerical constraints matching. Furthermore, numerical question-answering datasets, such as TAT-QA[22], typically emphasize reasoning and computational tasks within specific domains. These tasks, however, are not aligned with the goals of similarity-based retrieval systems, which evaluate document relevance based on semantic proximity rather than numerical computation.

To address this limitation, we create NumConQ to evaluate retrievals on numerical constraint queries across various domains. NumConQ includes semantically similar documents with varying numerical attributes, ensuring that models are tested on their ability to handle numerical constraints rather than relying solely on semantic cues. The process of dataset construction is shown in Fig. 2.

3.2.1 Dataset collection

To facilitate a thorough evaluation of retrievers across various numerical constraint scenarios, we source data from five distinct domains: healthcare, finance, education, sports, and movies. The specific datasets incorporated into our collection are as follows:

- **Diabetes dataset:** Health-related data focusing on demographics, medical history, and health metrics for diabetes research.
- **Leagues dataset:** Financial and organizational data from top global sports leagues, including revenue, salaries, and team statistics.
- **QS ranking dataset:** Global university rankings, detailed across academic and employer reputation, research impact, and international diversity.
- **Olympics dataset:** Comprehensive data on athletes and events in the Olympic Games history.
- **IMDB dataset:** Detailed movie information including release year, genre, ratings, and box office performance.

Each dataset incorporates a multitude of numerical constraints alongside other non-numerical attributes. The numerical values within these datasets range from single-digit figures to ten-digit numbers, reflecting the spectrum of numerical constraints encountered in everyday scenarios.

3.2.2 Document construction

Considering that the original datasets are primarily

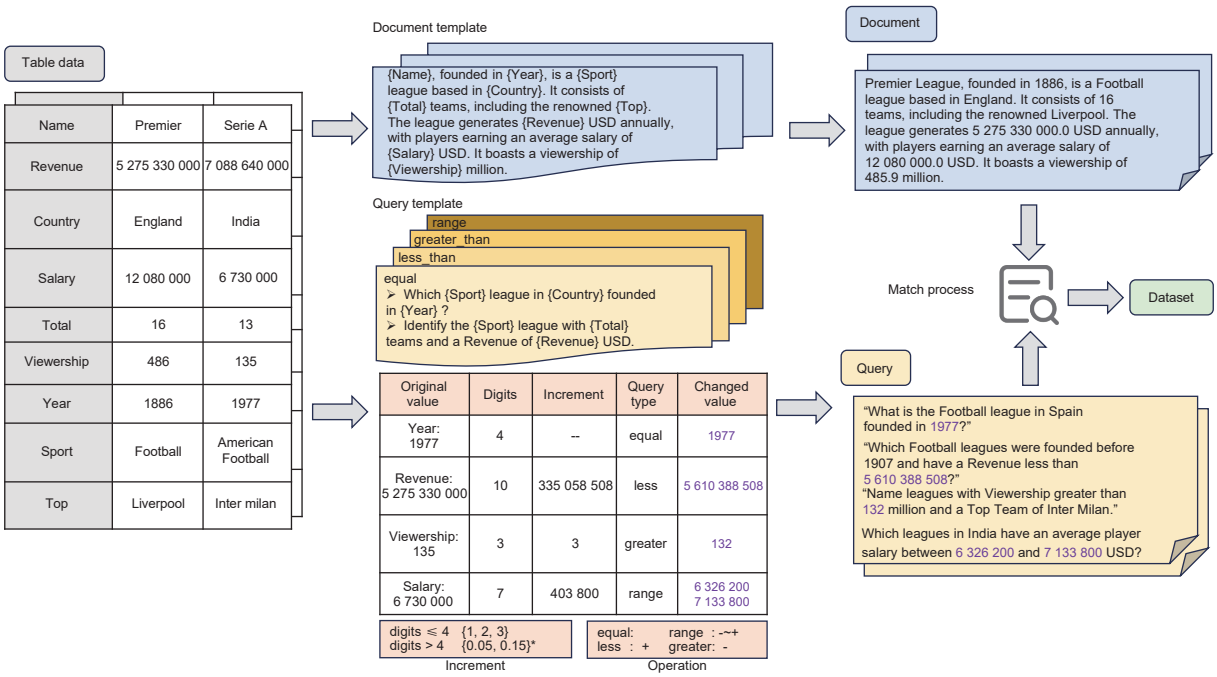


Fig. 2 Dataset construction process, including document construction, query generation, and test data preparation.

structured in tabular formats and inherently lack semantic information, we employ a template-based approach to transform each entry into natural language text. These generated texts are treated as independent documents, collectively forming the knowledge base that the retrievers will access.

To enhance semantic diversity and richness, we develop multiple templates for each dataset, which are generated and refined by LLMs. This variability in templates helps prevent the generation of monotonous and repetitive text, thereby providing a more robust evaluation framework for retrievers.

3.2.3 Query construction

In constructing queries, we distinguish between two primary types of numerical constraints: exact match and range match. To systematically generate these queries, we manually craft four different templates: Equal, Less_than, Greater_than, and Range. Similar to document construction, each template type is realized in multiple expression forms to enhance semantic diversity. For instance, within the Olympics dataset, we generate queries, such as "Which male athlete from Russia competed in the 2003 Winter Olympics at an age less than 25?" and "Find athletes whose height is between 179 and 185 centimeters and whose sex is female". These queries are designed to include at least one numerical constraint.

When generating range match queries, we apply specific transformations to the original numerical values extracted from the documents to ensure realism and plausibility. For numerical values with 1−4 digits (e.g., height, weight, year), we adjust values by subtracting 1−3 units. For example, to create a "Less_than" year query based on an original value of 2003, we might generate a constraint like "less than 2004" or "less than 2005". For numerical values exceeding four digits, which typically represent larger scales, the transformation is applied as a percentage of the original value, specifically between 5% to 15%. This approach ensures that the generated queries remain within plausible and meaningful ranges, thereby avoiding unrealistic constraints such as "before 2176" or "height is 270 cm".

To determine the relevance of documents in response to a given query, we extract the pertinent numerical and non-numerical data from each document. These extracted values are systematically compared against the constraints specified in the query. A document is deemed relevant if it satisfies all numerical constraints and any non-numerical constraints in the query. The determination of such relevance is achieved through a rule-based matching process, which ensures precision in its execution. To avoid the potential impact on precision and recall results due to significant differences in the number of documents corresponding to queries generated from different datasets, we limit the number of documents per query to no more than 10. Comprehensive statistics of NumConQ are shown in Table 1.

3.3 Experiment setting and metrics

In naive RAG[23], the retrieval process operates without the involvement of an LLM, as the retriever independently processes the query. The process begins by encoding the entire knowledge base using different embedding models. Subsequently, each numerical constraint query is encoded using the same embedding model to maintain consistency in the vector space representations. Cosine similarity scores between the encoded queries and documents are computed, and the top-k most relevant documents are selected as the retrieval set for each query.

To assess the performance of different dense embedding models within the RAG framework for numerical constraint query retrieval, we select five representative models based on three criteria: (1) proven effectiveness in retrieval tasks, (2) architectural diversity, and (3) research reproducibility.

JinaAI-v2 specializes in long-context numerical constraints through its 8000-token content window. Topic Aware Sampling-Balanced (TAS-B) optimizes real-time multi-constraint discrimination via topic-aware sampling. BGE-M3 supports multilingual numerical parsing with its hybrid sparse-dense

Table 1 Statistics of NumConQ. "#Avg. D/Q" indicates the average number of relevant documents per query, and "#Avg. Words" indicates the average number of words within the document.

Dataset	Number of queries	Number of documents	#Avg. D/Q	#Avg. Words
Diabetes	1512	1447	4.9	45.6
Leagues	1347	600	4.1	49.3
QS	1267	1366	3.3	82.5
Olympics	1229	1236	3.3	44.0
IMDB	1222	713	2.3	89.0
Total	6577	5362	3.6	60.8

architecture. Dense Passage Retrieval (DPR) provides baseline performance for established RAG systems, while Contriver offers domain-specific optimization for technical document retrieval.

Since the number of documents corresponding to each query does not exceed 10, we choose recall@10 and precision@10 as evaluation metrics.

3.4 Results and analysis

We compare the performance of the retrievers (e.g., Contriever) for numerical and non-numerical constraint queries in Fig. 3, and provide a detailed comparison of their performance in Figs. 4 and 5.

3.4.1 Do dense retrievers perform similarly on numerical constraints and non-numerical constraints?

To address this question, we conduct a comparative experiment using the Olympics dataset in conjunction with the Contriever model. The objective is to evaluate the dense model's retrieval capabilities for numerical and non-numerical constraint queries. To ensure uniform retrieval difficulty, we utilize exact match queries that encompass several numerical and non-numerical constraints. In our experimental setup, for each query, we analyze the top 10 retrieved documents and evaluate the consistency between the constraints specified in the queries and those presented in the retrieved documents. The consistency is quantified by calculating the ratio of the number of constraints in the retrieved documents that matched the query constraints

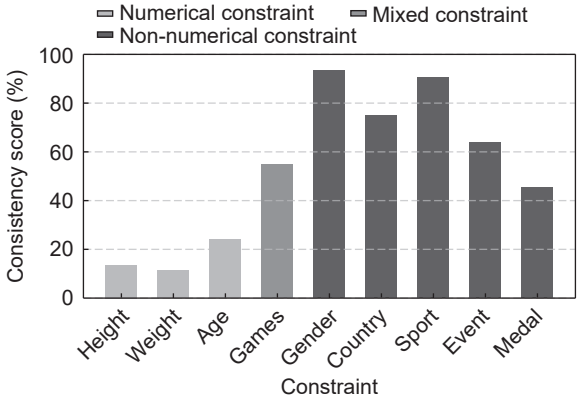
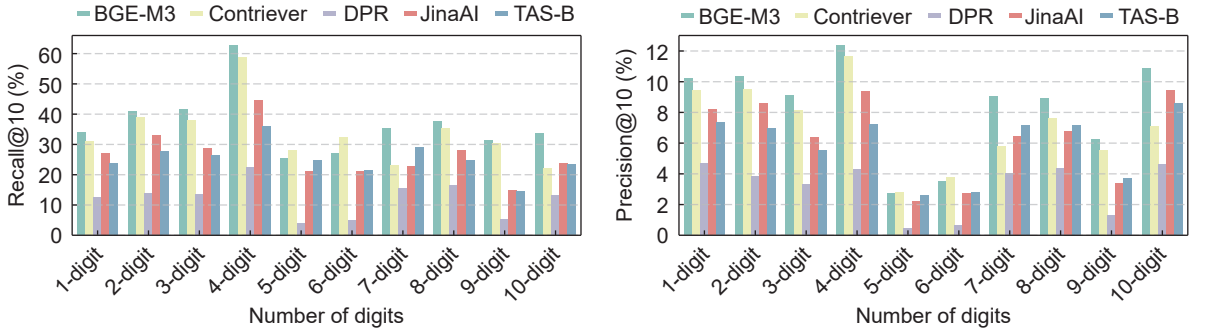
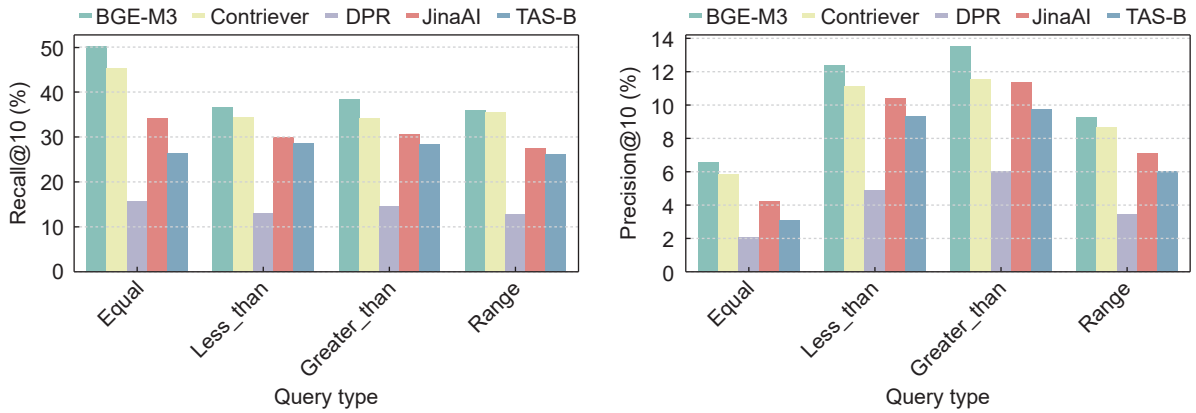


Fig. 3 Consistency scores for numerical and non-numerical constraints in the retrieved documents and the corresponding queries.



(a) Recall@10 across different digit number in queries (b) Precision@10 across different digit number in queries

Fig. 4 Recall@10 and Precision@10 for retrievers on NumConQ based on the number of digits in queries.



(a) Recall@10 on different query types (b) Precision@10 on different query types

Fig. 5 Recall@10 and Precision@10 for retrievers on NumConQ based on the query types.

to the total number of such constraints in the documents.

As illustrated in Fig. 3, numerical constraints such as Height, Weight, and Age exhibit low consistency scores, averaging 16.3%. In contrast, non-numerical constraints, like Gender, Country, Sport, Event, and Medal, achieve significantly higher consistency scores, averaging 73.64%. Additionally, we introduce mixed constraints, referred to as "Games", which combine numerical element representing the year and non-numerical element representing the season (e.g., 2024 Summer). The mixed constraint attains a consistency score of 59.42%, positioning it between numerical and non-numerical constraints in terms of retrieval performance. These findings demonstrate that dense retrievers are more effective at handling non-numerical constraints compared to numerical constraints.

3.4.2 Can dense retrievers effectively handle numerical constraint queries?

Based on the NumConQ dataset, our experimental findings suggest otherwise. Dense retrievers typically measure relevance by computing the distance between a query and documents, using metrics such as cosine similarity or dot product similarity[24]. In this experiment, we analyze performance from two perspectives: the number of digits in the query and the query type.

Figure 4 shows the results based on the number of digits in the queries. The experiment reveals that most retrievers perform best when the numerical constraints in the query contain four digits. However, even the best-performing model, BGE-M3, achieve Recall@10 and Precision@10 scores of 62.8% and 12.4%, respectively, which are far from satisfactory. Many retrievers display Recall@10 lower than 30% and Precision@10 under 10%. This indicates that, in general, dense retrievers struggle to process numerical constraint queries effectively. Additionally, in most cases, retrievers perform better on queries with 1-digit to 4-digit constraints than on those with more than 4 digits. This suggests that retrieval effectiveness is generally higher when handling lower-digit numbers than it is for higher-digit numbers, remaining challenges in ultra-long numerical processing. One contributing factor is that most training data for retrievers tend to include lower-digit values, which are more commonly encountered in real-world text. As a result, the learned embeddings may be better calibrated for smaller numerical ranges. Another critical aspect relates to tokenization: higher-digit numbers are often split into multiple tokens, fragmenting the numerical value and obscuring vital information, such as magnitude and positional context. This fragmentation can produce an incomplete or distorted representation of the numerical attribute, making it more challenging for the dense retriever to accurately assess similarity based on precise numerical criteria. The methodological considerations and potential remedies for this tokenization issue are discussed more extensively in Section 4.

Figure 5 presents the results based on the query type, showing that dense retrievers achieve better precision on "Range", "Less_than", and "Greater_than" queries compared to "Equal" queries. A likely explanation is that "Equal" constraints require stringent numerical matching, whereas dense retrievers predominantly rely on semantic similarity, making them relatively insensitive to the exact numerical value. In contrast, queries specifying ranges or inequalities accommodate a broader spectrum of relevant numerical values, thus aligning more closely with the approximate matching principles employed by dense embeddings[25]. Interestingly, the "Equal" type queries exhibit higher Recall@10, which we believe is also due to their strict numerical requirements: each query corresponds to a relatively small set of documents. As a result, when retrieving the top 10 documents, even a small number of relevant documents leads to a higher recall score.

3.4.3 Why do current dense retrievers perform poorly on numerical constraint queries?

A plausible explanation for the poor performance of dense retrievals in handling numerical constraint queries lies in their propensity to overemphasize certain semantic cues while neglecting numerical information. We refer to this phenomenon as attention bias. Due to such bias, retrievers may retrieve documents that appear semantically similar to the query, but do not actually satisfy the numerical conditions. In our constructed queries, both numerical and non-numerical constraints are present. When the query and document share the same non-numerical constraints, whether the document is genuinely relevant hinges on correct numerical matching. We find that many incorrectly retrieved documents show high similarity to the query's non-numerical information, suggesting an over-reliance on semantic overlap at the expense of accurate numerical constraint satisfaction.

By analyzing the training methods and datasets used

by various dense retrievers, we can partly explain the occurrence of attention bias. For example, Contriever is trained using a purely unsupervised method, where adjacent paragraphs are treated as positive samples. While this approach captures semantic relationships between adjacent text, it struggles to focus on specific numerical information[26]. DPR, which utilizes BM25[27] for negative sample mining, and TAS-B, with its topic-aware approach, both encounter a similar limitation. These methods primarily focus on semantic matching between textual content, often overlooking the explicit representation of numerical constraints. In contrast, BGE-M3 is trained with supervised data that incorporates both numerical and semantic information, which may explain why it performs better when handling numerical constraint queries.

Furthermore, most current dense retrievers rely on predominantly text-based datasets, which often overlook the semantic representation of numerical information. For example, the large-scale textual datasets used for training these models typically consist of books, web pages, or encyclopedic content. While these datasets perform well in natural language processing tasks, they lack structured annotations for numerical values, making it difficult for dense retrievers to effectively match and interpret numerical constraints.

In practice, users may provide only approximate numerical cues in their queries. Even when numerical ranges are specified, enumerating all potential values within that range is often impractical. Consequently, a retriever may overlook valid numerical values that are not explicitly mentioned. For example, a height of 178 cm is within the range of "between 176 and 181 cm," yet the query does not explicitly include "178".

4 Method

In this section, we propose a contrastive learning based fine-tuning strategy to enhance dense retrievers' sensitivity to numerical constraints. Furthermore, We identify inconsistencies in tokenizers' subword segmentation for numbers and propose a hybrid numerical representation method combining digital and word representations to mitigate this issue.

4.1 Method overview

Our proposed framework is illustrated in Fig. 6. To model negative samples at the semantic level, we employ in-batch negatives, while at the numerical level, we leverage hard negative mining tailored for numerical constraints. Through multi-granularity

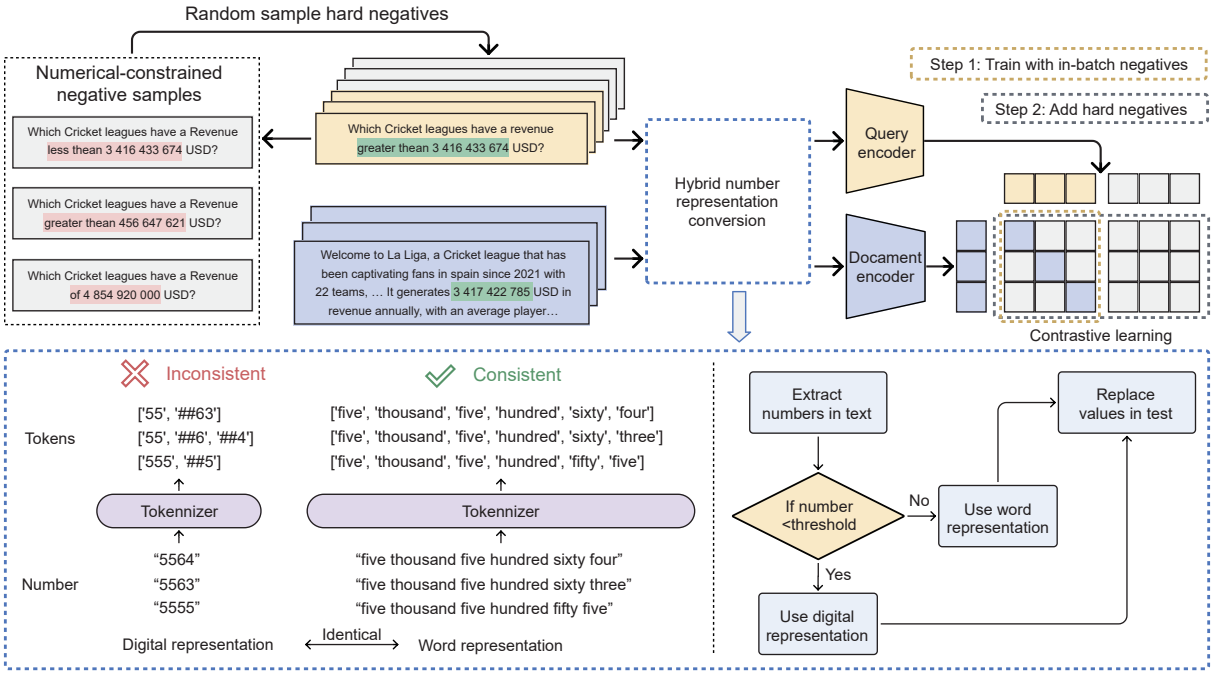


Fig. 6 Overview of our framework. In-batch negative sampling is firstly trained for general semantic adaptation. Then, hard negatives are incrementally incorporated, specifically created by altering numerical constraints in queries. Finally, tokenization issues of larger numbers are addressed through a hybrid strategy that retains small numbers in digital form but represents larger values in words.

contrastive learning, the model is capable of capturing both semantic and numerical information. Positive sample pairs consist of text-based queries with numerical and non-numerical constraints and documents that satisfy all the constraints of the queries. Hard negative samples are generated by modifying the numerical constraints in the positive sample queries, ensuring that the corresponding documents no longer meet the altered numerical constraints. Additionally, we identify inconsistencies in the subword tokenization of numbers with existing tokenizers. To address this issue, we propose a hybrid numerical representation method that combines digit-based and word-based representations to optimize the subword tokenization of numbers. Our approach integrates contrastive learning for numerical constraints with optimized representations for numerical data. Detailed descriptions are provided in subsequent sections.

4.2 Tokenization and challenges in numerical representation

Tokenization is a fundamental step in natural language processing, particularly for transformer-based models, like Bidirectional Enoceder Representations from Transformers (BERT)[28] and their derivatives, which are widely used in dense retrievers. Tokenization determines how input text is segmented into smaller units, influencing downstream model performance. Among popular methods, WordPiece tokenization plays a pivotal role in BERT-based dense retrievers due to its balance between vocabulary size and its ability to handle Out-Of-Vocabulary (OOV) words[29].

4.2.1 WordPiece tokenization

WordPiece[30] is a subword tokenization algorithm designed to address the trade-offs between vocabulary size and representation flexibility. By decomposing words into smaller subword units, WordPiece ensures that rare or domain-specific terms can be represented using combinations of more frequent subunits. The algorithm constructs a vocabulary by iteratively merging the most frequent character pairs in the training data, until a predefined vocabulary size is reached. However, when it comes to numbers, which are treated as sequences of characters rather than atomic units, WordPiece's subword-level segmentation can lead to significant inconsistencies. Other tokenization, such as SentencePiece[31], adopts the Unigram language model, which may be more flexible in segmenting numbers, but its vocabulary construction method may exacerbate the fragmentation of numerical semantics. BPE[32] is based on character frequency merging and lacks explicit modeling of numerical semantics. These tokenization methods have similar issues with WordPiece, and due to the widespread use of WordPiece, we employ it for deeper analysis of numbers tokenization.

4.2.2 Inconsistent tokenization of numbers

Numbers often undergo inconsistent segmentation under WordPiece tokenization, primarily due to the algorithm's reliance on statistical frequency rather than semantic or structural understanding of numbers. This inconsistency is evident in the examples provided in Table 2, where numerically similar inputs are tokenized differently. For instance, the number "5563" is segmented as ["55", "##6", "##3"], while "5564" is tokenized as ["55", "##64"]. These examples highlight the lack of a consistent representation even for numerically similar inputs.

4.2.3 Tokenization pattern analysis

Further, we conduct a statistical analysis of tokenization patterns across numbers of varying lengths. We define the consistency of tokenization patterns between two numbers of the same length as follows: if the tokenized representations of the two numbers have the same number of tokens and the corresponding tokens have the same character lengths, their tokenization patterns are considered consistent. For example, the tokenized structures of ["52", "##28"] and ["51", "##23"] are consistent, whereas ["55", "##6", "##3"] and ["55", "##64"] are inconsistent. We sample 1000 numbers for each digit number and analyzed their tokenization patterns. As shown in Fig. 7, the results reveal that the proportion of inconsistent tokenization patterns increases significantly with higher-digit numbers. This finding underscores the challenges posed by WordPiece tokenization in

Table 2 Tokenization results for different numbers using the WordPiece tokenizer.

Number	Token
5563	["55","##6", "##3"]
5564	["55", "##64"]
71 199	["71", "##19", "##9"]
71 200	["71", "##200"]
74 701	["747", "##01"]
74 702	["747", "##0", "##2"]
8 404 248	["840", "##42", "##48"]
8 594 254	["85", "##9", "##42", "##54"]

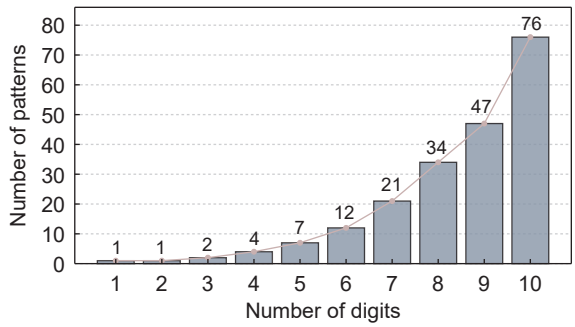


Fig. 7 Tokenization patterns of different digit numbers. The growth of inconsistent tokenization patterns with increasing digit numbers is illustrated, based on WordPiece tokenization of 1000 numbers per digit number.

preserving structural consistency for numerical inputs, particularly as their complexity grows.

The subword splits are determined by the relative frequency of substrings in the training corpus, leading to tokenization variability for numbers that differ only slightly. As a result, numerical semantics are not preserved systematically, and subtle numerical relationships may be obscured.

4.2.4 Implications for dense retrievers

The inconsistent tokenization of numbers introduces several challenges for dense retrieval systems, particularly in tasks involving numerical reasoning or constraints:

- **Loss of numerical semantics:** Tokenization inconsistencies result in semantic misalignment, where numerically similar numbers, like "5564" and "5563", are represented by distinct subword sequences. This misalignment prevents models from accurately capturing numerical proximity or patterns. And the number of digits is lost.
- **Difficulty in learning numerical relationships:** Since the tokenized representations of similar numbers differ significantly, the model struggles to learn numerical relationships effectively. This limits its ability to reason about numerical constraints or perform numerical comparisons.
- **Increased noise in training and retrieval:** Variability in numerical tokenization introduces noise during training, as the model must learn to generalize inconsistent subword representations. This hinders convergence and reduces the model's ability to generalize effectively to unseen numerical queries.

While WordPiece tokenization provides a robust framework for handling diverse textual inputs, its treatment of numbers presents a critical limitation for tasks requiring numerical reasoning. The inability to consistently represent similar numbers poses challenges for learning and retrieval in dense retrievers. Addressing these inconsistencies is essential for improving model performance on tasks involving numerical queries and constraints.

4.3 Hybrid numerical representation

To address the inconsistency of numerical tokenization, we propose a hybrid numerical representation method that combines digital and word representation. The core idea of this approach is to select different representation strategies based on the digit number of numbers, aiming to balance the performance of pre-trained models and the consistency of tokenization.

To determine the numerical magnitude threshold for retaining digital representation versus converting to word form, 15% of the training data are allocated as a validation set. The pre-trained base model's performance are evaluated across thresholds ranging from 0 to 10 000. As illustrated in Fig. 8, experimental results reveals that retrieval performance peaks at the threshold of 3000 with accuracy (ACC) of 15.40% and Mean Reciprocal Rank (MRR) of 0.266. Further expansion of the threshold range yields no additional performance gains.

This result suggests that the 3000 threshold sufficiently covers frequently occurring low-digit numbers in pre-training corpora, whose tokenization patterns and semantic stability have been thoroughly learned by the base model. So, we adopt direct digital representation for numbers with fewer digits (e.g., 1 to 3000), preserving the base model's strong representation of low-digit numbers. For numbers with higher digits (e.g., above 3000), we convert them into

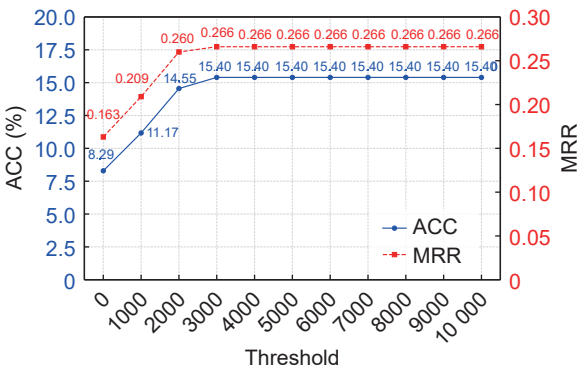


Fig. 8 Maximum ACC and MRR under different thresholds of hybrid numerical representation.

word representations. For instance, the number "5564" is transformed into "five thousand five hundred sixty four". This word representation ensures higher tokenization pattern consistency for high-digit numbers, as these words frequently appear in the training corpus and can be tokenized into independent tokens. Moreover, words, such as "thousand" and "hundred", effectively preserve the semantic information of digit positions, which is lost during direct tokenization of digital representation.

To implement this method, we use regular expressions to extract numbers (e.g., "1000", "5565", and "673 333") from the text during training and convert them into the corresponding representations based on a predefined threshold. During inference, instead of encoding the original text directly, we encode the text transformed by the hybrid numerical representation to generate embeddings for documents and queries. This approach aims to mitigate the inconsistency in tokenization patterns for high-digit numbers while preserving the capability of pre-trained models for low-digit numbers.

4.4 Contrastive learning for numerical semantics

Contrastive learning is a technique in unsupervised learning aimed at learning effective feature representations by distinguishing between positive and negative sample pairs. In the unsupervised setting, positive sample pairs are typically generated by applying data augmentation to anchor points, while negative samples are created by randomly sampling from a mini-batch of positive samples or a sampling pool[33]. By training a retriever in this manner, the model can effectively capture semantic information from text. For example, models, such as Contriever, leverage data augmentation methods like random cropping to construct positive text sample pairs, while negative samples are drawn from a queue containing elements from previous batches.

However, the reliance on unsupervised contrastive learning on data augmentation to generate positive sample pairs poses challenges, particularly when dealing with numerical queries. Augmentation techniques, such as random cropping or replacement, may disrupt the integrity of numerical values, making them unsuitable for tasks that involve precise numerical reasoning. Furthermore, negative samples generated through random sampling might not capture the nuanced distinctions required for such queries. To overcome these limitations, we adopt supervised contrastive learning[34] to handle numerical queries better. This approach allows us to directly incorporate numerical constraints from the text, providing a more targeted and robust way of constructing sample pairs.

As described in Section 3.2, we generate query-document pairs in textual form from structured data using templates, ensuring that these queries and documents are not part of the test set. Through metadata rule matching, we guarantee that documents in positive sample pairs meet the necessary query constraints, including both numerical and non-numerical conditions. For semantic-level negative samples, we utilize in-batch negative sampling, wherein negative samples are drawn from other positive sample pairs within the same mini-batch. For the generation of numerical-level negative samples, we mine hard negatives by modifying the numerical constraints in the queries. By adjusting only the numerical aspects of the queries, we ensure that the hard negative samples remain semantically similar, with the key difference being whether their numerical constraints are satisfied. This strategy underscores the importance of numerical constraints, as modifying queries rather than documents avoids introducing unrealistic data. In total, we construct a training dataset comprising 3760 samples, including both positive sample pairs and hard negative samples, to effectively train the model on numerical queries.

Recognizing the potential risks of premature introduction of hard negative samples, such as model overfitting to these challenging instances, we have incorporated the principles of curriculum learning into our training methodology. Our strategy is executed in phases: initially, during the fine-tuning process, we employ the in-batch negatives technique, facilitating rapid model adaptation to the task-specific data distribution. Subsequently, in the intermediate and final stages of fine-tuning, we introduce hard negative samples. This methodical escalation not only mitigates the risk of overfitting, but also progressively enhances the model's capacity to handle numerical constraints. Empirical results demonstrate that this phased training strategy substantially elevates the model's overall performance.

Training loss. During our training process, we employ the contrastive InfoNCE loss function[35]. Specifically, given an anchor document d, a relevant query q^+, and a set of irrelevant queries $\{q_i^-\}_{i=1, 2, ..., K}$,

the contrastive loss is defined as follows:

$$L(q^+, d) = \log\left(1 + \sum_{i=1}^{K} \exp\left(\frac{s_i^- - s^+}{\tau}\right)\right),$$
$$s^+ = f(d) \cdot f(q^+),$$
$$s_i^- = f(d) \cdot f(q_i^-) \quad (1)$$

where $f(\cdot)$ denotes the feature extraction function, s^+ denotes the similarity score between d and q^+, s_i^- denotes the similarity score between d and the i-th negative query, and $\tau \in \mathbf{R}^+$ denotes the temperature hyperparameter in the contrastive loss used to adjust the scale of similarity measurement between samples.

5 Evaluation

In this section, we evaluate the retrievers trained using our proposed method and conduct an in-depth analysis of the results. These models are specifically designed for numerical constraint queries, and the best-performing model is called NC-Retriever, short for numerical constraint-aware retriever.

5.1 Setting

Implementation. We implement all our experiments on a NVIDIA A800 GPU using Python 3.8, and Transformers 4.44.2. We employ the Contriever model as our foundational pre-trained model and fine-tune it using the open-source code provided by Izacard et al.[9] The models are trained for a total of 3968 steps with a batch size of 64, optimized using the AdamW optimizer and a learning rate of 0.0001, alongside a linear scheduler. This learning rate aligns with Contriever's configuration for stable fine-tuning. For each training instance, we include 1 hard negative. For our experiments, we train multiple model variants, incorporating different numerical representation methods and introducing hard negatives at various stages of training.

Baselines. For zero-shot retrievers, we select the five dense retrievers mentioned in Section 3.2 and add a widely-used sparse retriever, BM25, as baselines. For the fine-tuned baseline, we fine-tune Contriever, using only in-batch negatives as negative samples and the original digital representation for numbers. The remaining fine-tuning settings and hyperparameters are consistent with NC-Retriever.

Test datasets and metrics. We employ the benchmark dataset NumConQ established in Section 3.2. The evaluation of the retriever performance primarily utilizes two metrics: Recall@k and Precision@k. We also employ nDCG@10 and mAP@10 to provide a more comprehensive assessment.

5.2 Main result

The evaluation results underscore the superior performance of the NC-Retriever model in handling numerical constraint retrieval tasks, particularly when compared to other retrievers. As shown in Table 3, NC-Retriever significantly outperforms both the fine-tuned baseline and the state-of-the-art zero-shot retriever BGE-M3 across recall and precision at various ranks. Notably, NC-Retriever demonstrates remarkable improvements in recall@10 and precision@10, with

Table 3 Model recall and precision results on NumConQ. NC-Retriever refers to our optimal version that uses a hybrid numerical representation and introduces hard negatives starting from 70% of the total training steps. JinaAI refers to the jina-embeddings-v2-base-en model. The relative gain is calculated based on the performance of the NC-Retriever compared to the best baseline, with underlines used to annotate the best baseline. The values presented in bold font represent the results achieved by our proposed model, NC-Retriever.

(%)

Model	Recall@k						Precision@k					
	k=1	k=5	k=10	k=20	k=50	k=100	k=1	k=5	k=10	k=20	k=50	k=100
BM25	11.65	22.46	28.93	35.58	43.54	50.02	16.38	8.58	6.13	4.29	2.31	1.41
DPR	3.09	9.53	14.06	20.47	31.08	42.01	7.59	5.24	4.11	3.09	1.99	1.39
TAS-B	6.73	19.18	27.39	36.82	49.17	58.19	13.55	9.02	7.05	5.14	2.90	1.79
JinaAI	7.72	22.02	30.57	40.95	55.03	65.33	15.84	10.89	8.29	5.92	3.38	2.07
BGE-M3	13.03	30.40	40.37	51.39	65.67	74.64	23.41	14.11	10.45	7.25	4.03	2.41
Contriever	12.76	28.88	37.40	46.92	60.51	70.33	22.05	12.81	9.30	6.39	3.56	2.21
Contriever-finetuned	20.00	38.78	48.26	58.26	70.03	77.70	31.52	16.65	11.83	7.86	4.19	2.46
NC-Retriever	**31.37**	**55.70**	**66.95**	**77.16**	**87.07**	**92.11**	**52.29**	**26.88**	**18.63**	**11.86**	**5.76**	**3.16**
Relative gain	+56.85	+43.63	+38.73	+32.44	+24.33	+18.55	+65.89	+61.44	+57.48	+50.89	+37.47	+28.46

gains of 38.73% and 57.48%, respectively, compared to Contriever-finetuned. When compared to BGE-M3, NC-Retriever achieves improvements of 65.84% in recall@10 and 78.28% in precision@10.

These results suggest that NC-Retriever's focus on contrastive learning, tailored specifically for numerically constraint queries, is a key factor driving its enhanced performance. The incorporation of hard negatives starting from 70% of the total training steps, along with a hybrid numerical representation, enables the model to better capture the relationships between queries containing numerical constraints and documents with numerical information. For sparse retriever, BM25's poor performance stems from its reliance on exact term matching, which is incompatible with the variable tokenization of numbers and the need for magnitude-aware ranking. Furthermore, these substantial performance improvements highlight a gap in the pre-training and fine-tuning datasets of existing models, which typically lack sufficient numerical constraint data. This gap may explain why other models, such as Contriever and BGE-M3, struggle to achieve similar success in numerical constraint retrieval tasks. Despite the notable advancements, the results also suggest that there is still considerable room for improvement.

5.3 Comparison between different numerical representations

This subsection delves into the performance discrepancies among models utilizing different approaches for representing numbers. We compare three numerical representation methods: digital, word, and our proposed hybrid method. All models incorporate hard negative samples into training from the beginning. Table 4 compares these methods with four key performance indicators: nDCG@10, mAP@10, Recall@10, and Precision@10.

The data from Table 4 shows that the hybrid representation outperforms both the digital and word representations across all evaluated metrics. Specifically, the hybrid model achieves a Recall@10 of 63.95%, a Precision@10 of 17.50%, an nDCG@10 of 54.91%, and a mAP@10 of 47.39%. These results are better than the digital and word representations, confirming the efficacy of our hybrid approach in harnessing both representations' strengths to enhance overall performance.

Further analysis of model performance across different digit numbers, as shown in Fig. 9, sheds light on the adaptability of these representations. This detailed investigation reveals trends: for lower digit numbers (1-digit to 4-digit), models using digital

Table 4 Performance comparison on NumConQ of models with different numerical representation: nDCG@10, mAP@10, Recall@10, and Precision@10. The best results are highlighted in bold.

(%)

Model	Recall@10	Precision@10	nDCG@10	mAP@10
Digital	61.7	17.15	53.55	46.23
Word	61.61	16.36	50.45	42.64
Hybrid (ours)	**63.95**	**17.50**	**54.91**	**47.39**

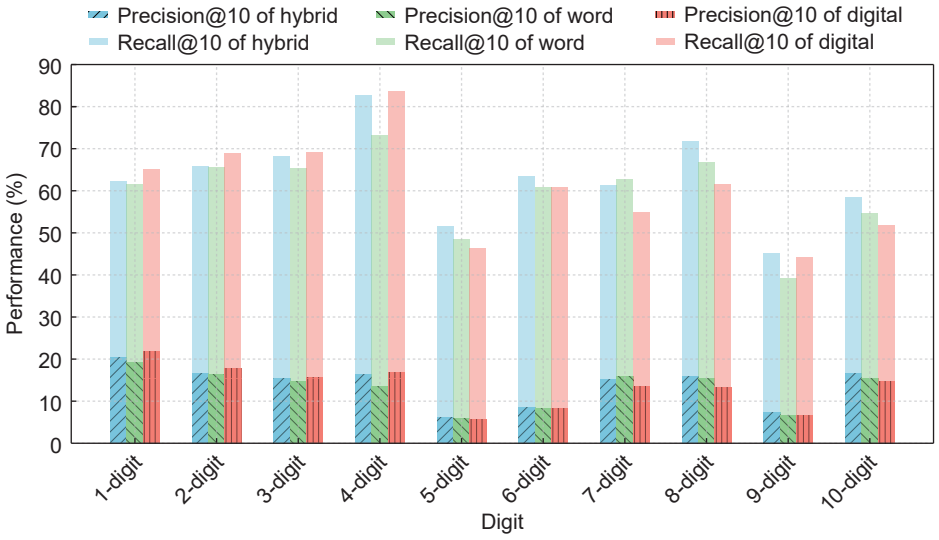


Fig. 9 Impact of numerical representation on model performance in NumConQ: Recall@10 and Precision@10 across digit number in queries.

representations outperform those using word representations in terms of Recall@10 and Precision@10. Conversely, as the digit number increases (5-digit to 10-digit), the performance of digital representation models declines relative to those using word representations. This observation supports our hypothesis that tokenization inconsistencies in digital representations adversely affect model performance as the numerical complexity increases, suggesting that the word representation, which treats digits as textual data, may handle complex numerical expressions more effectively than the digital representations. The hybrid representation, as intended, balances these trends. It closely follows the digital model's performance in lower digits, albeit slightly underperforming compared to pure digital representation, but still outperforming the word representation. Notably, in higher digit values, the hybrid model performs comparably or slightly better than the word model, effectively mitigating the drop in performance seen with the digital model. This balanced performance across all digit ranges highlights the hybrid model's effectiveness in leveraging the respective strengths of the digital and word representations, making it a robust choice for diverse numerical data processing tasks.

5.4 Impact of introducing hard negatives at different training stages

We evaluate the effects of introducing hard negatives that are constructed based on numerical constraints, analyzing the metrics of precision@10 and recall@10. The data summarized in Fig. 10 illustrates the performance across various training stages, with stages defined as a percentage of the total training steps.

Our results demonstrate that hard negatives are optimally introduced in the mid-to-late stages of training. For example, introducing hard negatives at training stage 0.7 results in the highest recorded precision and recall values of 18.63% and 66.95%, respectively. This improvement signifies that once the model is sufficiently familiar with the basic semantic landscape and can reliably differentiate between regular negatives and positives, it can effectively leverage the complexity introduced by hard negatives to refine its understanding and handling of nuanced numerical semantic distinctions.

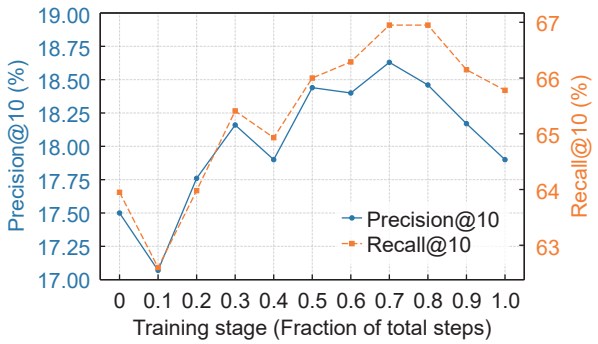


Fig. 10 Precision@10 and Recall@10 at different training stages with hard negatives. All models use hybrid numerical representation.

However, introducing hard negatives at the beginning of training (training stage 0) results in decreased model performance compared to stages where hard negatives are introduced later or not at all. At training stage 0, where hard negatives are introduced immediately, the model records a recall of 63.95% and a precision of 17.50%. This early introduction seems detrimental as the model struggles to differentiate between the closely semantically related hard negatives and positive samples.

Contrastingly, in training stage 1.0, where no hard negatives are used throughout the training, the model performance shows improved stability with a precision of 17.9% and a recall of 65.78%. This suggests that withholding hard negatives until the model has adequately learned to distinguish between regular negatives and positives can prevent initial confusion and stabilize the learning process.

6 Discussion

This section explores the generalization capabilities of models in handling numerical constraints and examines the evolution of numerical constraint embeddings before and after fine-tuning. We first investigate whether models fine-tuned on specific numerical constraints can generalize to unseen constraints, revealing that while absolute performance declines, relative advantages persist. Next, we analyze how fine-tuning affects the embedding of numerical constraints, demonstrating improved local semantic similarity for exact match queries but the limited distinction between comparison logic and numerical proximity for range-match queries. These findings highlight the potential and limitations of current dense retrievers in understanding and generalizing numerical constraints.

6.1 Generalization in contextual dependency of numerical constraint

Numerical constraints in queries are context-

dependent; for instance, the value "20" represents different constraints in contexts such as "price less than 20" versus "age less than 20". Given this variability, an important question arises: Can models fine-tuned on specific numerical constraints generalize to entirely new constraints? To investigate this, we initially create synthetic columns and corresponding data, and then construct queries and documents using the method outlined in Section 3.2, resulting in a test dataset referred to as the NumConQ (Unseen). This dataset features queries containing numerical constraints not seen in the fine-tuning dataset and documents with semantically related, unseen numerical data.

Table 5 provides the results for models on NumConQ (Unseen). Performance analysis in scenarios with unseen numerical constraints reveals a decrease in absolute performance but the preservation of relative advantages. Specifically, in known constraints, the Contriever model has a Precision@1 of 22.05%, while the NC-Retriever improves it to 52.29%, marking a 137.19% increase. In contrast, in unseen constraints, the improvement is from 8.79% to 13.78%, a 56.77% increase. This suggests that while the model struggles with completely adapting to new constraints, the learned sensitivity to numbers and comparative logic still provides an advantage.

6.2 Numerical constraint embeddings: Before and after fine-tuning

We investigate how the embedding of numerical constraints changes before and after fine-tuning by constructing four types of pure numerical constraints: Equal (e.g., "Which number is 10?"), Less_than (e.g., "Which number is lower than 20?"), Greater_than (e.g., "Which number is greater than 30?"), and Range (e.g., "Which number is between 15 and 25?"), with corresponding documents following the format "The number is X." (e.g., "The number is 10." or "The number is 20.").

We visualize the similarity matrices between these queries and documents using heatmaps, as shown in Fig. 11. For "Equal" queries, fine-tuned models (regardless of numerical representation) exhibit better properties compared to the base model, as documents with values closer to the query number show higher semantic similarity, though this property is only observed locally. For Less_than, Greater_than, and Range queries, the models capture the comparison logic only weakly and locally, as documents satisfying the comparison logic (e.g., values less than the query number) show higher similarity, but documents with values close to the query number (even if they do not satisfy the logic) also exhibit high similarity, indicating that the models struggle to distinguish between comparison logic and numerical proximity.

Table 5 Performance of models on NumConQ (Unseen). (%)

Model	Recall@k			Precision@k		
	k=1	k=10	k=100	k=1	k=10	k=100
Contriever	8.79	7.14	2.92	20.68	0.87	48.23
NC-Retriever	13.78	8.03	5.14	24.47	1.28	58.85

7 Conclusion and Limitation

In this paper, we set out to investigate how well widely used dense retrievers in RAG handle numerical constraint queries. To this end, we construct NumConQ, a multi-domain benchmark dataset of numerical constraint queries and found that, while existing retrievers excel at semantic matching, they often underperform in satisfying precise numerical requirements. We find that the underperformance stems from inconsistent tokenization of larger-digit numbers and an underemphasis on numerical attributes during training. To address these issues, we propose a two-phased contrastive learning framework that integrates hard negatives, as well as a hybrid numerical representation that balances tokenization consistency. Experimental results demonstrate that our approach significantly boosts recall and precision in numerical constraint retrieval, highlighting the value of specialized training methods for numerical data in RAG.

Despite these advancements, several limitations remain. First, the proposed framework has not been extensively tested on compound queries requiring multi-step numerical reasoning or dynamic, context-dependent attributes. Second, the strong emphasis on numerical constraints in our contrastive training may diminish the model's general semantic coverage, as the retriever reallocates capacity to numerical precision at the possible expense of broader contextual understanding. Finally, performance may degrade when handling multi-constraint queries that require logical operations. Addressing these areas in future work could further enhance the applicability and

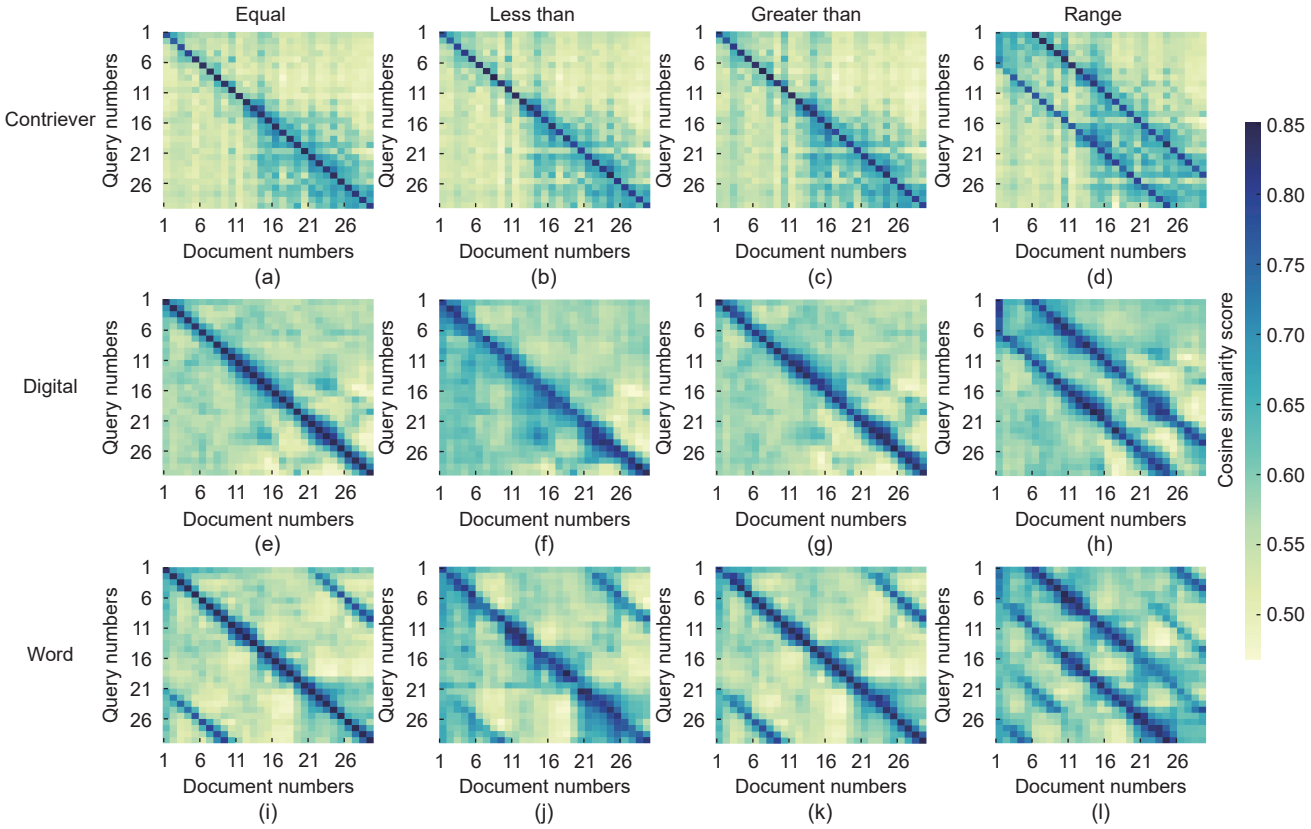


Fig. 11 Heatmap visualization of similarity scores for models across pure numerical constraints. Heatmaps depict similarity between queries (e.g., "Which number is 10?" or "Which number is lower than 20?") and documents (e.g., "The number is 10." or "The number is 20."). The value range is 1 to 30. Darker colors indicate higher similarity.

robustness of dense retrieval methods in real-world scenarios involving intricate constraint queries.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (Nos. 62276063, U23B2057, and 62176185), the Natural Science Foundation of Jiangsu Province (No. BK20221457), the Natural Science Foundation of Beijing Municipality (No. L247008), the Tongji University Innovative Design and Intelligent Manufacturing Discipline Group Project, Tongji University Construction Project of the National Artificial Intelligence Industry-Academia Collaborative Innovation Platform, and Tongji University 2023 Interdisciplinary Collaborative Research Project.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. T. Yih, T. Rocktäschel, et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, in *Proc. 34th Int. Conf. Neural Information Processing Systems*, Vancouver, Canada, 2020, p. 793.

[2] S. Kukreja, T. Kumar, V. Bharate, A. Purohit, A. Dasgupta, and D. Guha, Performance evaluation of vector embeddings with retrieval-augmented generation, in *Proc. 9th Int. Conf. Computer and Communication Systems*, Xi'an, China, 2024, pp. 333–340.

[3] Z. Liu, H. Zhang, C. Xiong, Z. Liu, Y. Gu, and X. Li, Dimension reduction for efficient dense retrieval via conditional autoencoder, in *Proc. 2022 Conf. Empirical Methods in Natural Language Processing*, Abu Dhabi, United Arab Emirates, 2022, pp. 5692–5698.

[4] J. Sun, H. Zhang, C. Lin, X. Su, Y. Gong, and J. Guo. APOLLO: An optimized training approach for long-form numerical reasoning, in *Proc. 2024 Joint Int. Conf. Computational Linguistics, Language Resources and Evaluation*, Torino, Italia, 2024, pp. 1370–1382.

[5] S. Pakhomov, S. Bjornsen, P. Hanson, and S. Smith, Quality performance measurement using the text of electronic medical records, *Med. Decis. Making*, vol. 28, no. 4, pp. 462–470, 2008.

[6] T. Kannampallil and J. Adler-Milstein, Using electronic health record audit log data for research: Insights from early efforts, *J. Am. Med. Inform. Assoc.*, vol. 30, no. 1, pp. 167–171, 2022.

[7] S. Xu, L. Pang, H. Shen, and X. Cheng, BERM: Training the balanced and extractable representation for matching

to improve generalization ability of dense retrieval, in *Proc. 61st Annu. Meeting of the Association for Computational Linguistics*, Toronto, Canada, 2023, pp. 6620–6635.

[8] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu. M3-Embedding: Multi-Linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, in *Proc. Association for Computational Linguistics*, Bangkok, Thailand, 2024, pp. 2318–2335.

[9] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, Unsupervised dense information retrieval with contrastive learning, *Trans. Mach. Learn. Res*., arxiv prepriat arXiv: 2112. 09118., 2022.

[10] S. Hofstätter, S. C. Lin, J. H. Yang, J. Lin, and A. Hanbury, Efficiently teaching an effective dense retriever with balanced topic aware sampling, in *Proc. 44th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, Virtual Event, 2021, pp. 113–122.

[11] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. T. Yih, Dense passage retrieval for open-domain question answering, in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing*, Virtual Event, 2020, pp. 6769–6781.

[12] M. Günther, J. Ong, I. Mohr, A. Abdessalem, T. Abel, M. K. Akram, S. Guzman, G. Mastrapas, S. Sturua, B. Wang, et al., Jina embeddings 2: 8192-token general-purpose text embeddings for long documents, arXiv preprint arXiv: 2310.19923, 2023.

[13] T. Chen, Y. Sun, Y. Shi, and L. Hong, On sampling strategies for neural network-based collaborative filtering, in *Proc. 23rd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Halifax, Canada, 2017, pp. 767–776.

[14] V. T. Ho, Y. Ibrahim, K. Pal, K. Berberich, and G. Weikum, Qsearch: Answering quantity queries from text, in *Proc. 18th Int. Semantic Web Conf. the Semantic Web*, Auckland, New Zealand, 2019, pp. 237–257.

[15] Z. Jia, P. Christmann, and G. Weikum, Faithful temporal question answering over heterogeneous sources, in *Proc. ACM on Web Conf*., Singapore, 2024, pp. 2052–2063.

[16] S. Han, H. Kang, B. Jin, X. Y. Liu, and S. Y. Yang, XBRL agent: Leveraging large language models for financial report analysis, in *Proc. 5th ACM Int. Conf. AI in Finance*, Brooklyn, NY, USA, 2024, pp. 856–864.

[17] A. Biswal, L. Patel, S. Jha, A. Kamsetty, S. Liu, J. E. Gonzalez, C. Guestrin, and M. Zaharia, Text2SQL is not enough: Unifying AI and databases with TAG, arXiv preprint arXiv: 2408.14717, 2024.

[18] L. Xiong, C. Xiong, Y. Li, K. F. Tang, J. Liu, P. N. Bennett, J. Ahmed, and A. Overwijk, Approximate nearest neighbor negative contrastive learning for dense text retrieval, in *Proc. 9th Int. Conf. Learning Representations*, Virtual Event, 2021, pp. 1–16.

[19] E. Khramtsova, S. Zhuang, M. Baktashmotlagh, and G. Zuccon, Leveraging LLMs for unsupervised dense retriever ranking, in *Proc. 47th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, Washington, DC, USA, 2024, pp. 1307–1317.

[20] X. Hu, Z. Shan, X. Zhao, Z. Sun, Z. Liu, D. Li, S. Ye, X. Wei, Q. Chen, B. Hu, et al., KaLM-Embedding: Superior training data brings a stronger embedding model, arXiv preprint arXiv: 2501.01028, 2025.

[21] A. Ahmad, N. Constant, Y. Yang, and D. Cer, ReQA: An evaluation for end-to-end answer retrieval models, in *Proc. 2nd Workshop on Machine Reading for Question Answering*, Hong Kong, China, 2019, pp. 137–146.

[22] F. Zhu, W. Lei, Y. Huang, C. Wang, S. Zhang, J. Lv, F. Feng, and T. S. Chua, TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance, in *Proc. 59th Annu. Meeting of the Association for Computational Linguistics and the 11th Int. Joint Conf. Natural Language Processing*, Virtual Event, 2021, pp. 3277–3287.

[23] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, Retrieval-augmented generation for large language models: A survey, arXiv preprint arXiv: 2312.10997, 2024.

[24] H. Steck, C. Ekanadham, and N. Kallus, Is cosine-similarity of embeddings really about similarity? in *Proc. ACM on Web Conf*., Singapore, 2024, pp. 887–890.

[25] W. Kong, S. Khadanga, C. Li, S. K. Gupta, M. Zhang, W. Xu, and M. Bendersky, Multi-Aspect dense retrieval, in *Proc. 28th ACM SIGKDD Conf. Knowledge Discovery and Data Mining*, Washington, DC, USA, 2022, pp. 3178–3186.

[26] Y. Lei, L. Ding, Y. Cao, C. Zan, A. Yates, and D. Tao, Unsupervised dense retrieval with relevance-aware contrastive pre-training, in *Proc. Association for Computational Linguistics*, Toronto, Canada, 2023, pp. 10932–10940.

[27] S. Robertson and H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, *Found. Trends® Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.

[28] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in *Proc. 2019 Conf. North American Chapter of the Association for Computational Linguistics*: *Human Language Technologies*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[29] A. Benamar, C. Grouin, M. Bothua, and A. Vilnat, Evaluating tokenizers impact on OOVs representation with transformers models, in *Proc. 13th Language Resources and Evaluation Conf*., Marseille, France, 2022, pp. 4193–4204.

[30] M. Schuster and K. Nakajima, Japanese and Korean voice search, in *Proc. 2012 IEEE Int. Conf. Acoustics, Speech and Signal Processing*, Kyoto, Japan, 2012, pp. 5149–5152.

[31] T. Kudo and J. Richardson, SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing, in *Proc. 2018 Conf. Empirical Methods in Natural Language Processing*: *System Demonstrations*, Brussels, Belgium, 2018, pp. 66–71.

[32] R. Sennrich, B. Haddow, and A. Birch, Neural machine translation of rare words with subword units, in *Proc. 54th Annu. Meeting of the Association for Computational Linguistics*, Berlin, Germany, 2016, pp. 1715–1725.

[33] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick,

Momentum contrast for unsupervised visual representation learning, in *Proc. 2020 IEEE/CVF Conf. Computer Vision and Pattern Recognition*, Seattle, WA, USA, 2020, pp. 9726–9735.

[34] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, Supervised contrastive learning, in *Proc. 34th Int. Conf. Neural Information Processing Systems*, Vancouver, Canada, 2020, p. 1567.

[35] A. van den Oord, Y. Li, and O. Vinyals, Representation learning with contrastive predictive coding, arXiv preprint arXiv: 1807.03748, 2018.



Meng Wang received the BEng degree from Sichuan University, China in 2012, and the PhD degree from Xi'an Jiaotong University, China in 2018. He is currently an associate professor at College of Design and Innovation, Tongji University, China. His current research interests include multi-modal knowledge graph, large language models, network embedding, SPARQL query language, and data mining.



Feifan Wu received the BEng degree from Sichuan University, China in 2022. He is currently a master student at Southeast University, China. His main research interests include large language model and retrieval-augmented generation.



Yisong Wang received the BEng degree from Southeast University of Artificial Intelligence, China in 2024. He is currently a master student at Tongji University, China. His main research interests include knowledge graph and large language model.